CST8509 Value Function Approx

Today's Agenda

- Review Quiz
- Value Function Approximation
- Add actors to Gazebo simulated worlds

Qlearning: example of Temporal Distance Learning

Monte Carlo: need to go all the way to the end of an episode, then can learn

Temporal Distance: time in RL is measured in steps

TD(0): Learn from looking **one** step ahead (state vs next-state):

```
qtable[state][action] = qtable[state][action] + alpha * (reward + gamma * max(qtable[next_state]) - qtable[state][action])
```

qtable: the table of action-values approximating the action-value function

state: the current state

action: the current action

alpha: step size

reward: reward received from taking action in state

gamma: discount factor

next_state: the state resulting from taking action in state

Value Function Approximation: Large Scale problems

So far we have constructed a qtable(state,action)

- value for every state/action pair
- What about
    - Backgammon: 10^{20} states
    - Computer Go: 10^{170} states
- Cannot represent that whole qtable explicitly

Value Function Approximation

Solution for large MDPs:
Estimate value function with function approximation

w is a vector of weights of a neural network

$\hat{v}(s, w) \approx v_{\pi}(s)$
or

$\hat{q}(s, a, w) \approx q_{\pi}(s, a)$

Generalize from seen states to unseen states

Update parameter w using MC or TD learning

Value function approximation and DQN

Q-Learning: learn Q, the action-value function q(s,a)

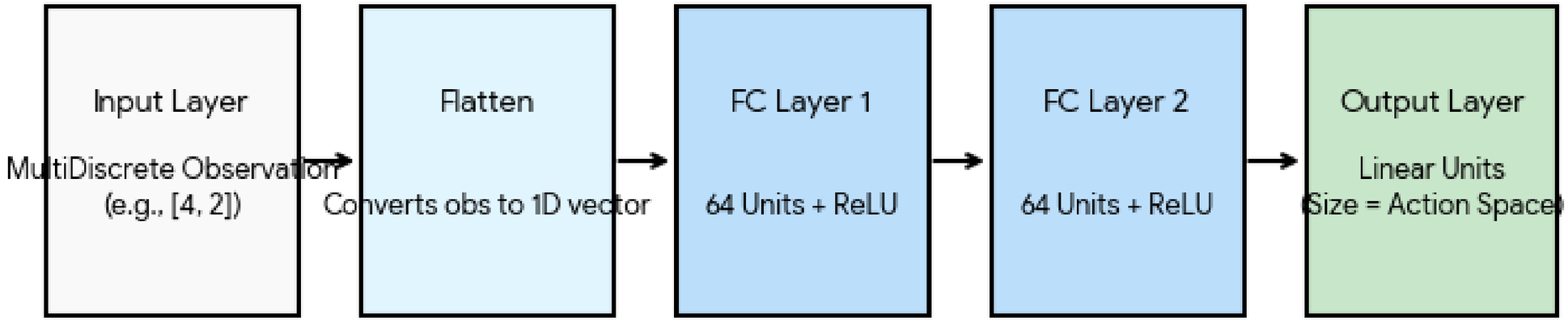
DQN: Deep Q-Network

DQN is similar to Q-learning except:

- No Q-table, we have Q-network instead
- Instead of updating Q-table, DQN trains Q-network
- Q-network implements an approximation of Q
- Input to Q-network is State, output is action values (same number of outputs as actions)

SB3 DQN Policy Architecture (MlpPolicy)

There are two of these neural networks: Online and Target:



DQN Online and Target Networks

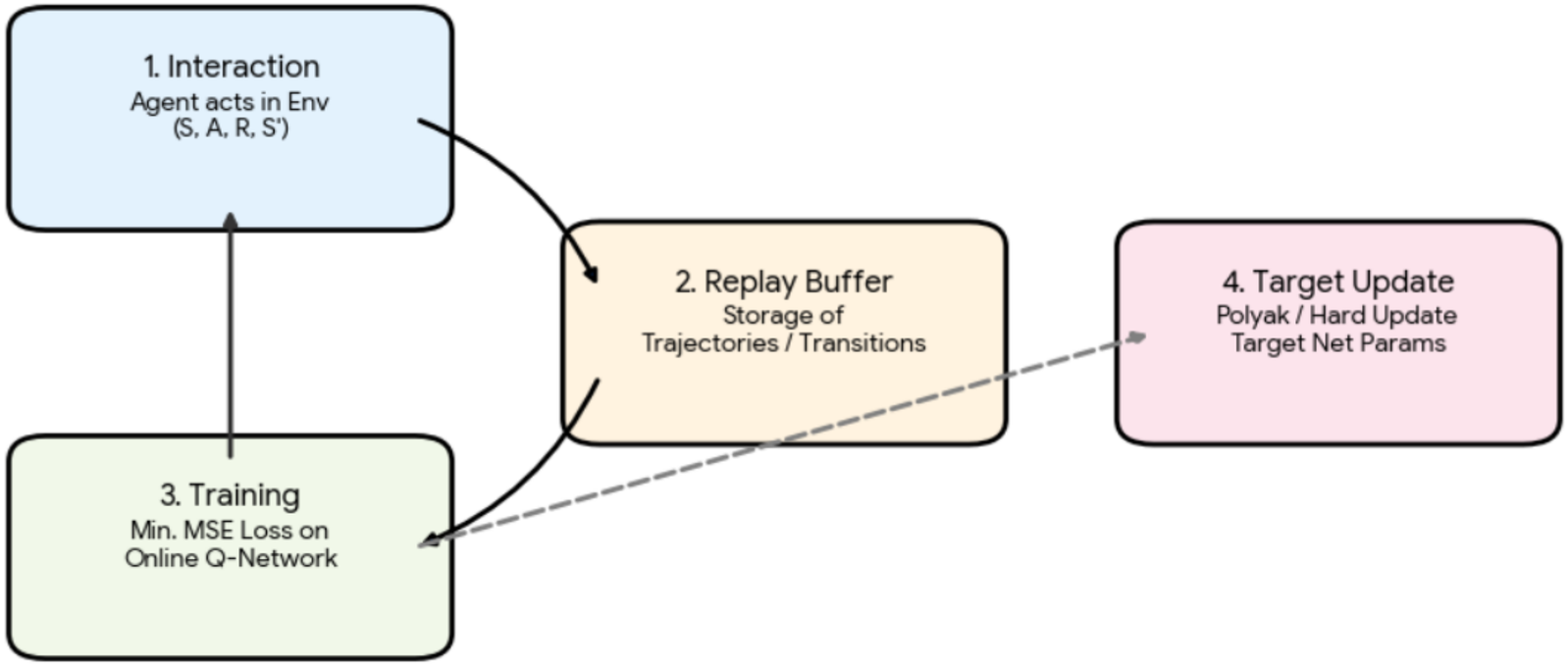
Online Network: Actively updated via gradients during every training step to minimize the loss function (MSE based on temporal difference (TD) error).

Target Network: A lagged copy of the online network used to provide stable Q-value targets. It is updated less frequently using a hard update or soft update (Polyak update) governed by the (small) tau parameter.

DQN Loss Function

$$L = E[(r + \gamma * max\, Q_{target(s',a')} - Q_{online(s,a)})^2]$$

DQN Algorithm



DQN training

Interaction: Agent collects (s, a, r, s') transitions from the environment. “learning_starts” are done before training

Replay Buffer: Transitions are stored to break temporal correlations.

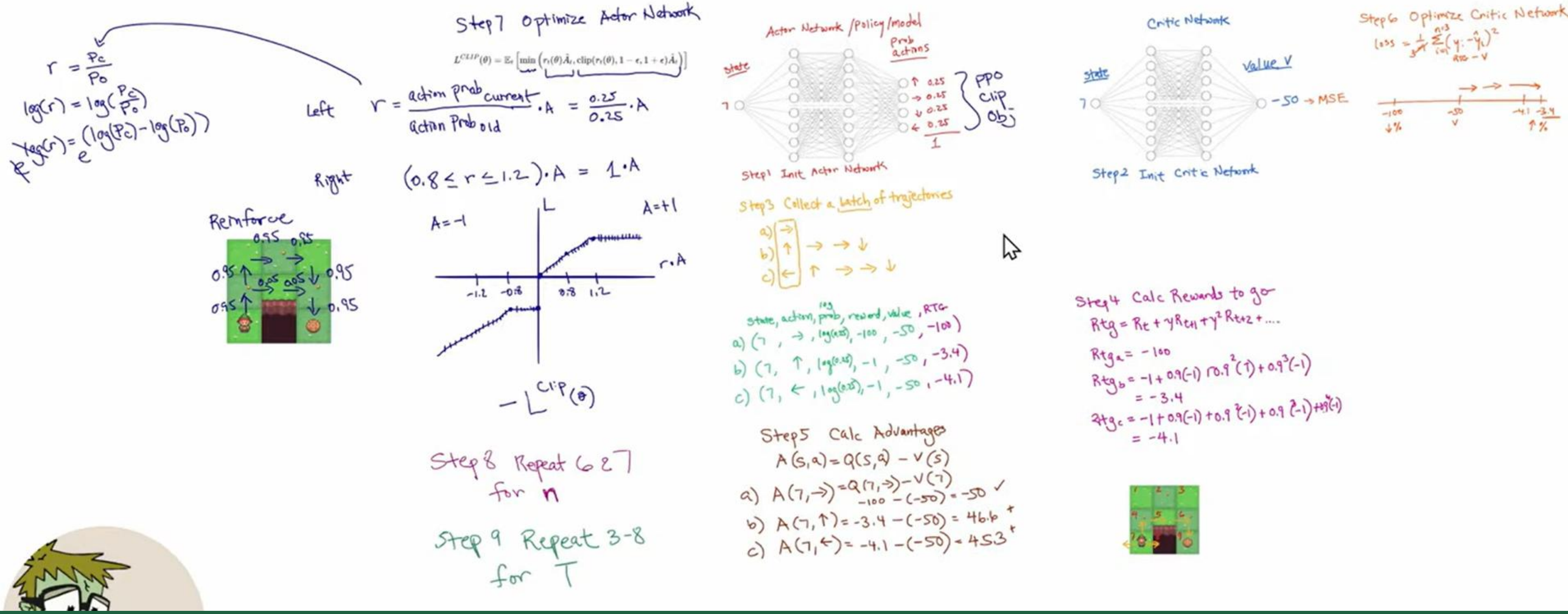
Optimization: Online Q-Network trains on random batches from the buffer. Done by default every 4 interactions (after learning_starts)

Target Update: Weights are copied to the Target Network for stability. Done every 10000 steps by default.

PPO and value function approximation

- PPO has and Actor Network and a Critic Network (Value function)

https://www.youtube.com/watch?v=5VHLd9eCZ-w



Actors in Gazebo worlds

Details here: https://gazebosim.org/docs/harmonic/actors

Skeletal animation: arms and legs move (for example)

Trajectory animation: actor moves in world

Red Ball to follow

Simple box following trajectory can be adapted to red ball:

https://classic.gazebosim.org/tutorials?tut=actor

```
<actor name="animated_ball">
      <link name="link">
        <visual name="visual">
          <geometry>
            <sphere>
              <radius>.2</radius>
            </sphere>
          </geometry>
          <material name="red">
            <ambient>1 0 0 1</ambient>
            <diffuse>1 0 0 1</diffuse>
            <specular>0 0 0 0</specular>
            <emissive>0 0 0 1</emissive>
          </material>
        </visual>
      </link>
```

Red Ball trajectory

```
<script>
        <loop>true</loop>
        <delay_start>0.000000</delay_start>
        <auto_start>true</auto_start>
        <trajectory id="0" type="square">
          <waypoint>
            <time>0.0</time>
            <pose>1 -1 1 0 0 0</pose>
          </waypoint>
          <waypoint>
            <time>1.0</time>
            <pose>1 1 1 0 0 0</pose>
          </waypoint>
          <waypoint>
            <time>2.0</time>
            <pose>1 1 1 0 0 0</pose>
          </waypoint>
          <waypoint>
            <time>3.0</time>
            <pose>1 -1 1 0 0 0</pose>
          </waypoint>
          <waypoint>
            <time>4.0</time>
            <pose>1 -1 1 0 0 0</pose>
          </waypoint>
        </trajectory>
      </script>
</actor>
```

Add to AWS small house

Can place the code for the red ball actor in

~/create3_ws/src/aws-robomaker-small-house-world/worlds/small_house.world

Let's also try adding the humanoid actor from the café world

Reinforcement Learning /w red ball

Similar to having the create3 follow the index finger

Have create3 follow red ball in simulation

Assignment 2:

- need gymnasium environment
  - This environment creates ROS 2 node
  - Step function publishes Twist message to cmd_vel
  - Return reward and new state (image or just offset?)